



CS-6418: Advanced Database Management System

M.Tech Exam-Ready Notes (Units I–V) — with GFG Reference Links & Table of Contents

How to use this file: Every topic opens with a one-line definition (write this first in the exam), followed by an explanation in plain English (what to *write* in your answer), then diagrams/examples/tables as supporting evidence. Each topic also has a  **GFG Reference** line — click it to read the GeeksforGeeks article for that exact topic before/after revising the note.

Note on the GFG links: Links marked [\[Verified\]](#) were checked and open the exact GFG article. Links marked [\[GFG Search\]](#) open a GeeksforGeeks *search results* page for that topic (used where I couldn't confirm one single canonical URL) — click the top result there; this avoids sending you to a dead/guessed link.



Table of Contents

- [Course Outcomes](#)
- [Syllabus Map](#)
- **UNIT I: DBMS Fundamentals & ER Model**
 - [1.1 Advantages of the DBMS Approach](#)
 - [1.2 Views of Data & Data Independence](#)
 - [1.3 Schema and Sub-Schema](#)
 - [1.4 Data Models](#)
 - [1.5 Database Languages](#)
 - [1.6 Transaction Management \(Preview\)](#)
 - [1.7 DBA, Users & Data Dictionary](#)
 - [1.8 Database Architectures](#)
 - [1.9 ER \(Entity-Relationship\) Model](#)
 - [Types of Attributes](#)
 - [Design Issues in ER Modeling](#)
 - [Mapping Cardinalities](#)
 - [Keys in the ER Model](#)
 - [Weak vs Strong Entity Sets](#)
 - [Specialization, Generalization, Aggregation, Inheritance](#)
 - [Design of ER Schema — Steps](#)
 - [Reduction of ER Schema to Tables](#)
 - [1.10 Domains, Relations, Types of Relations](#)
- **UNIT II: Relational Algebra, Relational Calculus & SQL**
 - [2.1 Structure of the Relational Model](#)
 - [2.2 Relational Algebra](#)
 - [2.3 Relational Calculus](#)
 - [2.4 Query-By-Example \(QBE\)](#)
 - [2.5 SQL — Basic Structure](#)
 - [Set Operations](#)
 - [Aggregate Functions](#)

- Null Values
 - Nested Subqueries
 - Derived Relations
 - Views
 - Modification of the Database (DML)
 - Join Relations (SQL JOIN types)
 - DDL in SQL
- **UNIT III: Functional Dependencies, Normalization & Database Integrity**
 - 3.1 Functional Dependencies (FD)
 - Closure of Attributes (X^+)
 - Closure of FDs & Armstrong's Axioms
 - Irreducible (Minimal) Set of FDs
 - 3.2 Introduction to Normalization
 - The Three Anomalies
 - Lossless Decomposition
 - FD Diagram
 - 3.3 Normal Forms
 - 1NF · 2NF · 3NF
 - Dependency Preservation
 - BCNF
 - 4NF (Multivalued Dependency)
 - 5NF (Join Dependency)
 - 3.4 Database Integrity
 - Domain Rules
 - Attribute Rules
 - Assertions
 - Triggers
 - Integrity & SQL (ON DELETE/UPDATE)
- **UNIT IV: Transactions, Concurrency & Recovery**
 - 4.1 Transactions & ACID Properties
 - Transaction States
 - Implementation of Atomicity & Durability
 - 4.2 Concurrent Execution
 - Problems from Uncontrolled Concurrency
 - 4.3 Serializability
 - 4.4 Concurrency Control
 - 4.5 Deadlock
 - 4.6 Failure Classification
 - 4.7 Storage Structure Types & Stable Storage
 - 4.8 Data Access (Buffer Management)
 - 4.9 Log-Based Recovery
 - Deferred Database Modification
 - Immediate Database Modification & Checkpoints
 - 4.10 Query Processing and Optimization
- **UNIT V: Graph, Spatial, Temporal Databases & NoSQL**
 - 5.1 Graph Databases

- [5.2 Spatial Databases](#)
- [5.3 Temporal Databases](#)
- [5.4 NoSQL Databases](#)
-  [Quick Exam-Day Revision Checklist](#)

Course Outcomes (COs)

CO	Description
CO1	Build a strong foundation of query languages through relational algebra, calculus & QBE
CO2	Design conceptual, logical & physical database models through ER model and normalization
CO3	Develop SQL proficiency on simple & advanced features — concurrency, transactions, recovery in a multiuser environment
CO4	Acquire necessary skills for NoSQL-based database application development
CO5	Exposure to graph, spatial, and temporal databases

Syllabus Map

Unit	Hours	Focus
I	8	DBMS fundamentals, architecture, ER Model, keys
II	8	Relational Algebra, Relational Calculus, SQL
III	8	Functional Dependencies, Normalization, Database Integrity
IV	8	Transactions, Concurrency Control, Recovery, Query Processing
V	—	Graph, Spatial, Temporal Databases & NoSQL

[↑ Back to Table of Contents](#)

UNIT-I: DBMS Fundamentals & ER Model

1.1 Advantages of the DBMS Approach

Definition to open with: *A Database Management System (DBMS) is software that enables the creation, storage, retrieval, and management of data in a structured way, providing a controlled interface between users/applications and the underlying data.*

Explain by contrasting with the older file-processing system (this contrast is the actual exam answer):

Problem in File-Processing Systems	How DBMS Solves It
Data redundancy & inconsistency — same data duplicated across multiple files, updates to one copy	Centralized data storage — one copy, all applications access the same data

Problem in File-Processing Systems	How DBMS Solves It
don't reflect in others	
Difficulty in accessing data — need to write a new program for every new type of query	Query languages (SQL) allow ad-hoc querying without new programs
Data isolation — data scattered in files of different formats, hard to write programs that combine data	Unified data model brings all data together logically
Integrity problems — constraints (e.g., "balance must be positive") scattered across application code, easy to miss in a new program	Integrity constraints are defined once, centrally, and enforced by the DBMS itself
Atomicity problems — a failure midway through an operation (e.g., a fund transfer) can leave data in an inconsistent state	Transaction management with ACID properties guarantees atomicity (Unit IV)
Concurrent access anomalies — multiple users modifying data simultaneously can cause inconsistency	Concurrency control mechanisms (locking, timestamping — Unit IV)
Security problems — hard to enforce who can see/change what data at file level	DBMS provides authorization/access-control at the level of relations, views, even individual attributes

Exam tip: For "advantages of DBMS", list all 7 points above briefly — this is one of the most repeated theory questions.

 **GFG Reference [Verified]:** [Advantages of DBMS over File System](#)

[↑ TOC](#)

1.2 Various Views of Data & Data Independence

Three-Schema Architecture (also called ANSI/SPARC Architecture)

Definition: The three-schema architecture separates the user's view of the database from the physical storage details, through three levels of abstraction.

EXTERNAL LEVEL (View level)

→ Multiple user-specific views; each user sees only the part of the database relevant to them

CONCEPTUAL LEVEL (Logical level)

→ Describes WHAT data is stored and the relationships among them (the full logical structure, e.g., tables, columns, constraints) — same for all users

INTERNAL LEVEL (Physical level)

→ Describes HOW data is physically stored — file organization, indexing, data structures on disk

Explain each level in one line:

- **Physical/Internal level** — the lowest level; describes how data is actually stored on disk (indexes, file structures, block sizes).
- **Logical/Conceptual level** — describes the overall logical structure of the whole database (tables, relationships, constraints) — this is the level a database administrator works at.
- **View/External level** — the highest level; describes only the part of the database that a specific application or user needs to see (e.g., a clerk sees only salary-related columns, not medical records).

Data Independence

Definition: *Data independence is the capacity to change the schema at one level of the database without having to change the schema at the next higher level.*

Type	Definition
Physical Data Independence	The ability to modify the <i>physical/internal</i> schema (e.g., change file organization, add an index) without needing to change the <i>conceptual</i> schema or application programs.
Logical Data Independence	The ability to modify the <i>conceptual</i> schema (e.g., add a new table/attribute) without needing to change the <i>external</i> views or application programs that don't use the changed part.

Exam tip: Logical data independence is considered *harder to achieve* than physical data independence, because application programs are usually more tightly tied to the logical structure of the data than to its physical storage — this comparison is a common 2-mark question.

 **GFG Reference** [[GFG Search](#)]: [Three-Schema Architecture & Data Independence in DBMS](#)

[↑ TOC](#)

1.3 Schema and Sub-Schema

- **Schema** — the overall logical design/structure of the entire database (the "blueprint"), which rarely changes.
- **Instance** — the actual data stored in the database *at a particular moment in time* (changes constantly as data is inserted/updated/deleted).
- **Sub-schema** — a subset of the schema, representing the specific portion of the database that is relevant/visible to a particular user or application (corresponds to the External Level above).

Analogy to write in exam: Schema is like the *design of a house* (blueprint) — it stays the same. Instance is like the *furniture currently in the house* — it changes over time even though the design doesn't.

 **GFG Reference** [[GFG Search](#)]: [Instance and Schema in DBMS](#)

[↑ TOC](#)

1.4 Data Models

Definition: A data model is a collection of conceptual tools for describing data, data relationships, data semantics, and data constraints.

Types of Data Models:

- └ Relational Model → data organized as tables (relations) of rows and columns [most widely used today]
- └ Entity-Relationship Model → data described via entities, attributes, and relationships (used for design, Section 1.9)
- └ Object-based Model → data as objects with attributes and methods (Object-Oriented DB)
- └ Semi-structured Model → data with flexible/self-describing structure, e.g., XML, JSON, NoSQL document model (Unit V)
- └ (Legacy) Hierarchical & Network Models → tree/graph-structured, mostly of historical interest today

 **GFG Reference** [[GFG Search](#)]: [Data Models in DBMS](#)

[↑ TOC](#)

1.5 Database Languages

Definition: Database languages are used to define, manipulate, and control data in a DBMS.

Language	Full Form	Purpose	Example Commands
DDL	Data Definition Language	Defines/modifies the database schema (structure)	CREATE , ALTER , DROP , TRUNCATE
DML	Data Manipulation Language	Manipulates the actual data stored	SELECT , INSERT , UPDATE , DELETE
DCL	Data Control Language	Controls access/permissions to data	GRANT , REVOKE
TCL	Transaction Control Language	Manages transactions	COMMIT , ROLLBACK , SAVEPOINT

Explain further: DDL statements are compiled and stored in the **data dictionary** (Section 1.7 below), which the DBMS consults before executing any DML statement, to verify the request is consistent with the schema.

 **GFG Reference** [[GFG Search](#)]: [DDL, DML, DCL and TCL in SQL](#)

[↑ TOC](#)

1.6 Transaction Management (Preview — full detail in Unit IV)

Definition: A transaction is a logical unit of work that consists of one or more database operations, which must be executed as an *all-or-nothing* unit to maintain database consistency.

Brief mention here (details in Unit IV): The transaction manager ensures **ACID** properties (Atomicity, Consistency, Isolation, Durability) even when the system crashes mid-way or multiple transactions run concurrently.

 **GFG Reference [Verified]:** [ACID Properties in DBMS](#) (full detail in [Section 4.1](#))

[↑ TOC](#)

1.7 Database Administrator (DBA) & Users, Data Dictionary

Database Administrator (DBA)

Definition: The DBA is the person (or team) who has central control over both the data and the programs that access it, responsible for the overall management of the database system.

Functions of a DBA (list all 5 for full marks):

1. **Schema definition** — creates the original database schema by writing DDL statements.
2. **Storage structure and access method definition** — decides how data is physically stored and indexed.
3. **Schema and physical organization modification** — alters the schema as requirements evolve.
4. **Granting authorization for data access** — controls which users/roles can access which parts of the data (DCL).
5. **Routine maintenance** — periodic backups, ensuring adequate disk space, monitoring performance, and handling recovery from failures.

Data Dictionary (also called System Catalog)

Definition: The data dictionary is a special set of tables, maintained by the DBMS itself, that stores **metadata** — "data about the data" — such as table names, column names/types, constraints, indexes, and user permissions.

Explain why it matters: Every time a query is submitted, the DBMS consults the data dictionary first to check that the tables/columns referenced actually exist and that the constraints are respected — this is why the data dictionary is sometimes called the "brain" of the DBMS.

 **GFG Reference [GFG Search]:** [Functions of DBA and Data Dictionary in DBMS](#)

[↑ TOC](#)

1.8 Database Architectures

Centralized vs Client-Server vs Distributed

1. Centralized Architecture

→ All data + DBMS software resides on a SINGLE machine/server; users connect via terminals

2. Client-Server Architecture



→ Clients send requests (SQL queries) over a network; the server processes them and returns results.

→ Two-tier: client talks directly to DB server.

→ Three-tier: client → application server (business logic) → database server (adds a middle layer for scalability, security, and reusable business logic – most common in modern web applications).

3. Distributed Architecture

→ Data is physically stored across MULTIPLE, possibly geographically separated, sites/servers,

but the system presents it to users as if it were a single unified database.

Exam tip: A common question is "Two-tier vs Three-tier architecture" — the key point to state is that the three-tier architecture adds an *application server* layer between the client and the database, which centralizes business logic, improves security (clients never talk to the DB directly), and makes the system easier to scale.

 **GFG Reference** [\[GFG Search\]: DBMS Architecture — 1-tier, 2-tier, 3-tier](#)

[↑ TOC](#)

1.9 ER (Entity-Relationship) Model

Definition: The Entity-Relationship (ER) model is a high-level conceptual data model used to describe the data, relationships, and constraints of an application in a way that is easy for non-technical stakeholders to understand, typically represented visually as an **ER diagram**.

 **GFG Reference** [\[GFG Search\]: Introduction of ER Model](#)

Basic Concepts

Concept	Definition	ER Diagram Symbol
Entity	A real-world object or concept that has independent existence and can be distinctly identified (e.g., a specific <i>Student</i> , <i>Employee</i>)	Rectangle

Concept	Definition	ER Diagram Symbol
Entity Set	A collection of entities of the same type sharing the same attributes (e.g., all <i>Students</i>)	Rectangle
Attribute	A property/characteristic that describes an entity (e.g., <i>name</i> , <i>roll_no</i>)	Oval
Relationship	An association between two or more entities (e.g., a <i>Student</i> <i>enrolls in</i> a <i>Course</i>)	Diamond
Relationship Set	A collection of relationships of the same type	Diamond

Types of Attributes (a very common exam diagram/list question)

1. Simple (Atomic) Attribute → cannot be divided further, e.g., *roll_no*
2. Composite Attribute → can be divided into sub-parts, e.g., *Name*
→ (*First Name*, *Last Name*)
3. Single-valued Attribute → holds only one value per entity, e.g., *date_of_birth*
4. Multi-valued Attribute → can hold multiple values, e.g., a person can have several *phone_numbers*
(represented with a double oval)
5. Derived Attribute → its value can be calculated/derived from other attributes,
e.g., *age* can be derived from *date_of_birth*
(represented with a dashed oval)
6. Key Attribute → uniquely identifies each entity in the entity set (underlined in diagrams)

 **GFG Reference** [[GFG Search](#)]: [Types of Attributes in ER Model](#)

Design Issues in ER Modeling

Explain these as the "decisions" a designer must make when building an ER model:

1. **Use of entity sets vs. attributes** — deciding whether something should be modeled as its own entity (e.g., *Address* as a separate entity) or as just an attribute of an existing entity, depending on whether it needs its own identity/relationships.
2. **Use of entity sets vs. relationship sets** — deciding whether a concept is best represented as an entity or as a relationship between existing entities (e.g., is "Enrollment" an entity or a relationship between *Student* and *Course*?).
3. **Binary vs. n-ary relationships** — most relationships are binary (between two entities), but some situations genuinely need a ternary (three-way) relationship (e.g., *Supplier-Part-Project*).
4. **Placement of relationship attributes** — deciding whether an attribute belongs to a participating entity or to the relationship itself (e.g., *date_of_enrollment* belongs to the *enrolls in* relationship,

not to Student or Course alone).

 **GFG Reference** [[GFG Search](#)]: [ER Model Design Issues](#)

Mapping Cardinalities (Mapping Constraints)

Definition: Mapping cardinality (or cardinality ratio) expresses the number of entities from one entity set that can be associated with entities from another entity set via a relationship.

1. One-to-One (1:1) → e.g., a Person has exactly one Passport, and a Passport belongs to exactly one Person
2. One-to-Many (1:M) → e.g., one Department has many Employees, but each Employee belongs to only one Department
3. Many-to-One (M:1) → the reverse view of 1:M — many Employees belong to one Department
4. Many-to-Many (M:N) → e.g., a Student can enroll in many Courses, and a Course can have many Students

Participation Constraints (often asked alongside cardinality):

- **Total participation** — every entity in the entity set must participate in at least one relationship instance (shown by a *double line* connecting the entity to the relationship diamond). E.g., every Loan must be associated with at least one Customer.
- **Partial participation** — some entities may not participate in any relationship instance at all (shown by a *single line*).

 **GFG Reference** [[GFG Search](#)]: [Mapping Cardinalities in DBMS](#)

Keys in the ER Model

Key Type	Definition
Super Key	Any set of one or more attributes that, taken collectively, can uniquely identify an entity in an entity set.
Candidate Key	A <i>minimal</i> super key — no proper subset of it is also a super key (removing any attribute breaks uniqueness).
Primary Key	The candidate key <i>chosen by the database designer</i> as the principal means of uniquely identifying entities.
Alternate Key	Candidate keys that were not chosen as the primary key.
Foreign Key	An attribute (or set of attributes) in one relation that refers to the primary key of another (or the same) relation, used to represent relationships between tables.

Exam tip: A classic conceptual question: "Every candidate key is a super key, but not every super key is a candidate key" — explain this using the *minimality* condition above.

 **GFG Reference [Verified]:** [Types of Keys in Relational Model \(Candidate, Super, Primary, Alternate and Foreign\)](#)

Weak vs. Strong Entity Sets

Definition:

- **Strong Entity Set** — an entity set that has its own primary key, i.e., it can be uniquely identified purely by its own attributes.
- **Weak Entity Set** — an entity set that does **not** have sufficient attributes to form its own primary key; it depends on a **strong (identifying/owner) entity set** for its identification. Its uniqueness comes from combining a **partial key** (also called a discriminator) with the primary key of the owner entity.

Diagram notation:

Weak entity set → double-lined rectangle

Its partial key → dashed underline

Identifying relationship → double-lined diamond

Example:

STRONG entity: Employee (primary key: emp_id)

WEAK entity: Dependent (partial key: dependent_name — not unique on its own)

→ Dependent is only uniquely identified as (emp_id, dependent_name) together,

because two different employees could each have a dependent named "Raj".

 **GFG Reference [GFG Search]:** [Weak Entity Set in DBMS](#)

Specialization, Generalization, Aggregation, Inheritance

These four are commonly asked together as "Advanced ER concepts" — explain each with an example:

1. **Specialization** — a **top-down** design process: start with one general entity set and divide it into several specialized sub-entity-sets based on distinguishing characteristics. Example: **Employee** specialized into **Manager** and **Engineer**.
2. **Generalization** — a **bottom-up** design process: start with several entity sets that share common features and combine (synthesize) them into a single, higher-level generalized entity set. Example: **Car** and **Truck** generalized into **Vehicle**.
3. **Aggregation** — a design abstraction where a **relationship set** is treated as a higher-level entity, so that it can itself participate in another relationship — used when a relationship needs to have relationships of its own. Example: a (**Employee**, **Project**) relationship called **Works_On** is aggregated so it can participate in a further relationship **Monitors** with a **Manager** entity.
4. **Inheritance** — in specialization/generalization, lower-level entity sets **inherit** all the attributes and relationship participations of their higher-level entity set, just like inheritance in OOP.

Explain the difference between Specialization and Generalization clearly (frequently confused, common exam question): Specialization starts from the *general* and moves to the *specific* (top-down); Generalization

starts from the *specific* and moves to the *general* (bottom-up) — they are essentially inverse processes of each other.

 **GFG Reference [Verified]:** [Generalization, Specialization and Aggregation in ER Model](#)

Design of ER Schema — General Steps

1. Identify the entities and entity sets relevant to the application.
2. Identify the relationships between entity sets.
3. Identify attributes for each entity and relationship, and classify them (simple/composite, single/multi-valued, derived).
4. Determine cardinality ratios and participation constraints for each relationship.
5. Identify primary keys for each entity set, and handle weak entity sets separately.
6. Apply specialization/generalization/aggregation where needed to handle hierarchies or complex relationships.
7. Draw the complete ER diagram and review it against the application's requirements.

Reduction of an ER Schema to Tables (Rules — very frequently asked, memorize all 6)

Definition: This is the process of converting an ER diagram into a set of relational tables (relational schema), which is a mandatory step before implementing the database in a relational DBMS.

1. **Strong entity set** → becomes a table whose columns are the entity's simple/derived attributes; the entity's primary key becomes the table's primary key. A composite attribute is represented by including its simple component attributes directly; a multi-valued attribute becomes a **separate table** containing the entity's primary key plus the multi-valued attribute.
2. **Weak entity set** → becomes a table that includes all of its own attributes, **plus the primary key of its owning (strong) entity set as a foreign key**; the table's primary key is the combination of the owner's primary key and the weak entity's partial key.
3. **1:1 relationship** → the primary key of either one of the two participating entities can be included as a foreign key in the other's table (commonly placed on the side with total participation, if any).
4. **1:M relationship** → the primary key of the entity on the "1" side is included as a foreign key in the table of the entity on the "many" side.
5. **M:N relationship** → requires a **separate junction/bridge table**, containing the primary keys of *both* participating entities as foreign keys (together forming the composite primary key of the new table), plus any descriptive attributes of the relationship itself.
6. **Specialization/Generalization** → can be mapped using one of three common methods: (a) a single table for the entire hierarchy with a "type" discriminator column, (b) separate tables for each sub-class containing only their own specific attributes plus the inherited primary key (used with total specialization), or (c) separate tables for each sub-class that duplicate all inherited attributes as well.

 **GFG Reference [GFG Search]:** [Converting ER Diagram to Tables in DBMS](#)

[↑ TOC](#)

1.10 Domains, Relations, and Types of Relations

Definition: A **domain** is the set of allowable/legal values that an attribute can take (e.g., the domain of **age** might be integers between 0 and 150).

Definition: A **relation** is a mathematical table — formally, a subset of the Cartesian product of a list of domains — used in the relational model to represent both entities and relationships as rows (tuples) and columns (attributes).

Properties of a relation (list these — common short-answer question):

- Each cell (row-column intersection) contains a single **atomic** value (no multi-valued or composite values — this is the requirement for 1NF, see Unit III).
- Each column has a distinct name (attribute name), and all values in a column come from the same domain.
- The order of rows and columns is immaterial (a relation is a *set* of tuples).
- No two rows (tuples) in a relation are identical.

Kinds of relations:

- **Base relation** — an actual, physically-stored relation/table defined by a **CREATE TABLE** statement.
- **Derived relation / View** — a "virtual" relation computed on demand from one or more base relations, typically defined by a query (**CREATE VIEW**) — it is not separately stored (see [Unit II, Section 2.5 Views](#)).

Types of Keys — Recap in Relational Terms

Already covered under [Section 1.9's ER Keys](#) — the same definitions of Super Key, Candidate Key, Primary Key, Alternate Key, and Foreign Key apply directly to relational tables, and this is the most commonly repeated set of definitions across the entire DBMS syllabus (appears in Unit I, II, and III questions alike). **Make sure you can write all five definitions from memory without hesitation.**

 **GFG Reference** [[GFG Search](#)]: [Relational Model in DBMS](#)

[↑ TOC](#)

UNIT-II: Relational Algebra, Relational Calculus & SQL

2.1 The Structure of the Relational Model

Definition: The relational model represents data and relationships among data as a collection of **relations (tables)**, each consisting of **tuples (rows)** and **attributes (columns)**, with an associated **relational schema** describing the table's structure (name + attributes + domains).

Basic terms to define together (common 5-mark "define the following" question):

- **Tuple** — a single row of a relation, representing one record.
- **Attribute** — a column of a relation.
- **Degree** — the number of attributes (columns) in a relation.
- **Cardinality** — the number of tuples (rows) in a relation.

- **Relation Schema** — the name of the relation plus the list of its attributes, e.g., `Student(roll_no, name, cgpa)`.
- **Relation Instance** — the actual set of tuples in the relation at a given point in time.

 **GFG Reference** [[GFG Search](#)]: [Relational Model — Concept of Keys](#)

[↑ TOC](#)

2.2 Relational Algebra

Definition: Relational algebra is a **procedural query language** consisting of a set of operations that take one or two relations as input and produce a new relation as output — it describes *how* to obtain a result, step-by-step.

 **GFG Reference** [[Verified](#)]: [Introduction of Relational Algebra in DBMS](#)

Fundamental (Basic) Operations

Operation	Symbol	Meaning	Example
Select	σ (sigma)	Selects rows (tuples) satisfying a given predicate	$\sigma(\text{cgpa} > 8.0)$ (Student)
Project	π (pi)	Selects specific columns (attributes), removing duplicates	$\pi(\text{name}, \text{cgpa})$ (Student)
Union	$\cup$	Combines tuples from two <i>union-compatible</i> relations, removing duplicates	$\pi(\text{name})(\text{Student}) \cup \pi(\text{name})(\text{Alumni})$
Set Difference	$-$	Tuples in the first relation but not in the second (union-compatible relations)	$R - S$
Cartesian Product	$\times$	Combines every tuple of one relation with every tuple of another (produces all possible combinations)	$\text{Student} \times \text{Course}$
Rename	ρ (rho)	Renames a relation and/or its attributes, useful for self-joins or clarity	$\rho(S, \text{Student})$

Union-compatibility rule (important, often asked): Two relations R and S are union-compatible if and only if (a) they have the same number of attributes (same degree), and (b) the domain of each corresponding attribute is the same. This condition is required for **Union**, **Intersection**, and **Set Difference**.

Extended (Derived) Operations

Explain that these can all be derived from the fundamental operations, but are provided for convenience:

Operation	Symbol	Meaning
Intersection	$\cap$	Tuples that appear in both relations — derivable as $R - (R - S)$

Operation	Symbol	Meaning
Join (Theta Join)	$\bowtie_{\theta}$	Combines tuples from two relations that satisfy a given condition θ — equivalent to $\sigma_{\theta}(R \times S)$
Equijoin	$\bowtie$ (with $=$)	A theta join where the condition uses only equality comparisons
Natural Join	$\bowtie$	An equijoin on all attributes with the same name in both relations, with the duplicate column automatically removed from the result
Division	$\div$	Used for queries like "find X that are related to all Y" — e.g., "find students who have enrolled in all courses"
Outer Join	$\bowtie_{\text{L}} \bowtie_{\text{R}} \bowtie_{\text{F}}$	Like a join, but retains unmatched tuples from one or both sides, filling in NULLs for missing values (Left/Right/Full Outer Join)

Explain Natural Join vs Theta Join vs Equijoin clearly (a favourite comparison question):

- **Theta Join** — the most general form; joins two relations based on *any* comparison condition ($<$, $>$, $=$, etc.), not necessarily equality.
- **Equijoin** — a specific theta join where the condition is restricted to equality ($=$) only.
- **Natural Join** — an equijoin performed automatically on *all* attributes with matching names in both relations, and — unlike a plain equijoin — it removes the duplicate copy of the joining attribute(s) from the final result.

Modification of a Database (Relational Algebra)

```

Insertion:   $r \leftarrow r \cup E$            (add the tuples from expression E into
relation r)
Deletion:    $r \leftarrow r - E$            (remove the tuples matching expression E
from r)
Update:      $r \leftarrow \pi(\text{attributes with modification})(\sigma(\text{condition})(r))$  — modeled
as a combination
            of a generalized projection over a selected subset of tuples

```

Explain: Relational algebra models insert/delete/update as expressions that produce a *new* relation which then replaces the old value of r — this is conceptually cleaner than the imperative "modify in place" thinking of SQL.

[↑ TOC](#)

2.3 Relational Calculus

Definition: Relational calculus is a **non-procedural (declarative)** query language — instead of specifying *how* to compute the result (like relational algebra), it specifies *what* result is wanted, using logical predicates/formulas.

Two forms of relational calculus (define both — a standard question):

1. **Tuple Relational Calculus (TRC)** — queries are expressed as $\{t \mid P(t)\}$, meaning "the set of all tuples t such that predicate $P(t)$ is true", where t ranges over *tuples* of a relation.
 - Example: $\{t \mid t \in \text{Student} \wedge t.\text{cgpa} > 8.0\}$ — find all student tuples with cgpa greater than 8.0.
2. **Domain Relational Calculus (DRC)** — similar to TRC, but the variables range over individual **domain values (attribute values)** rather than whole tuples: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$.
 - Example: $\{ \langle n, c \rangle \mid \exists r (r \in \text{Student} \wedge r.\text{name} = n \wedge r.\text{cgpa} = c \wedge c > 8.0) \}$.

Relational Algebra vs Relational Calculus (extremely common comparison — write this table in exam):

Relational Algebra	Relational Calculus
Procedural — specifies the sequence of operations to obtain the result	Non-procedural (declarative) — specifies only what result is required, not how to compute it
Uses operators like σ , π , $\cup$, $\times$, $\bowtie$	Uses predicate logic / first-order logic formulas
Considered <i>operational</i> , closer to how a query gets executed internally	Considered a <i>formal specification</i> of a query, closer to how SQL itself is conceptually designed
Both are proven to be equivalent in expressive power — every relational algebra expression can be converted to an equivalent calculus expression, and vice versa (this equivalence is called relational completeness)	

[↑ TOC](#)

2.4 Query-By-Example (QBE)

Definition: QBE is a **visual, non-procedural** query language where the user expresses a query by filling in example values and conditions directly into a skeleton table displayed on screen, rather than typing textual query syntax.

Explain the mechanism: The user is shown blank table "skeletons" matching the schema; to query, they type example constants (to specify exact matches) or example variables prefixed with an underscore (to represent values to be displayed/linked across tables), plus special operators (like **P.** to *print* a column in the output). The system translates this visual specification internally into a formal query.

 **GFG Reference** [**GFG Search**]: [Query By Example \(QBE\) in DBMS](#)

[↑ TOC](#)

2.5 SQL — Basic Structure

Definition: SQL (Structured Query Language) is the standard language for defining, manipulating, and querying relational databases — combining DDL, DML, DCL, and TCL in one language.

 **GFG Reference** [[GFG Search](#)]: [SQL Tutorial \(GeeksforGeeks\)](#)

Basic SELECT Structure

```
SELECT column1, column2, ...  
FROM table1, table2, ...  
WHERE condition  
GROUP BY column  
HAVING group_condition  
ORDER BY column;
```

Explain the conceptual (logical) order of execution — a commonly tested subtlety:

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
```

Even though **SELECT** is written first syntactically, it is logically evaluated *after* **FROM**, **WHERE**, **GROUP BY**, and **HAVING** — this is why you cannot directly use a column alias defined in **SELECT** inside the same query's **WHERE** clause.

Set Operations

```
SELECT name FROM Student  
UNION  
SELECT name FROM Alumni;  
  
SELECT name FROM Student  
INTERSECT  
SELECT name FROM Alumni;  
  
SELECT name FROM Student  
EXCEPT -- called MINUS in Oracle  
SELECT name FROM Alumni;
```

Note: **UNION** removes duplicates by default; **UNION ALL** retains them (and is faster since it skips the duplicate-elimination step).

Aggregate Functions

```
SELECT COUNT(*), AVG(cgpa), MAX(cgpa), MIN(cgpa), SUM(credits)  
FROM Student;
```

```
SELECT dept, AVG(cgpa)
FROM Student
GROUP BY dept
HAVING AVG(cgpa) > 8.0;
```

Explain WHERE vs HAVING (a very common exam question): WHERE filters individual rows **before** grouping takes place, and cannot use aggregate functions; HAVING filters **entire groups** *after* GROUP BY has been applied, and is specifically meant to use aggregate function conditions (like `AVG(cgpa) > 8.0`).

 **GFG Reference** [[GFG Search](#)]: [SQL WHERE vs HAVING Clause](#)

Null Values

Explain: NULL represents an unknown or missing value. Any arithmetic operation involving NULL yields NULL. Comparisons with NULL using = or <> yield UNKNOWN (not TRUE/FALSE), which is why SQL provides the special predicates IS NULL and IS NOT NULL to test for it explicitly.

Nested Subqueries

```
-- Subquery in WHERE with IN
SELECT name FROM Student
WHERE dept IN (SELECT dept FROM Department WHERE hod = 'Dr. Sharma');

-- Subquery with EXISTS
SELECT name FROM Student s
WHERE EXISTS (SELECT 1 FROM Enrollment e WHERE e.roll_no = s.roll_no);

-- Correlated subquery – the inner query references the outer query's table
SELECT name FROM Student s
WHERE cgpa > (SELECT AVG(cgpa) FROM Student WHERE dept = s.dept);
```

Explain "correlated subquery" specifically (frequently asked): A correlated subquery is one whose inner query depends on a value from the outer query (referencing a column from the outer table) and therefore must be logically re-evaluated **once for every row** processed by the outer query — unlike an ordinary (non-correlated) subquery, which is evaluated only once, independently of the outer query.

 **GFG Reference** [[GFG Search](#)]: [SQL Correlated Subquery](#)

Derived Relations

```
SELECT dept_avg.dept, dept_avg.avgcgpa
FROM (SELECT dept, AVG(cgpa) AS avgcgpa FROM Student GROUP BY dept) AS dept_avg
WHERE dept_avg.avgcgpa > 8.0;
```

Explain: A derived relation (or **inline view**) is a subquery placed in the **FROM** clause, which produces a temporary named relation that can then be queried like any regular table for the rest of that single query.

Views

```
CREATE VIEW HighCgpaStudents AS
SELECT name, dept, cgpa
FROM Student
WHERE cgpa > 8.5;

SELECT * FROM HighCgpaStudents;  -- queried just like a normal table
```

Definition: A view is a **virtual table** defined by a stored query; it does not itself store data (unless it's a *materialized view*), but is recomputed from the underlying base tables every time it is queried.

Why use views (list 3 reasons — common question):

1. **Security** — a view can expose only specific columns/rows of a sensitive table, hiding the rest from certain users.
2. **Simplicity** — a complex multi-table join can be wrapped in a view so users write simple **SELECT * FROM view** queries instead of repeating the join logic.
3. **Logical data independence** — if the underlying table structure changes, only the view definition needs updating, and applications querying the view remain unaffected.

 **GFG Reference** [[GFG Search](#)]: [SQL Views](#)

Modification of the Database (SQL DML)

```
INSERT INTO Student (roll_no, name, cgpa) VALUES (101, 'Rohit', 9.1);

UPDATE Student SET cgpa = 9.3 WHERE roll_no = 101;

DELETE FROM Student WHERE cgpa < 5.0;
```

Join Relations (SQL JOIN types)

```
-- INNER JOIN — only matching rows from both tables
SELECT s.name, e.course_id
FROM Student s INNER JOIN Enrollment e ON s.roll_no = e.roll_no;

-- LEFT OUTER JOIN — all rows from the left table, matched rows from the
right (NULL if no match)
SELECT s.name, e.course_id
FROM Student s LEFT OUTER JOIN Enrollment e ON s.roll_no = e.roll_no;

-- RIGHT OUTER JOIN — all rows from the right table, matched rows from the
left
```

```
SELECT s.name, e.course_id
FROM Student s RIGHT OUTER JOIN Enrollment e ON s.roll_no = e.roll_no;

-- FULL OUTER JOIN — all rows from both tables, NULLs where there's no
match on either side
SELECT s.name, e.course_id
FROM Student s FULL OUTER JOIN Enrollment e ON s.roll_no = e.roll_no;
```

Diagram to draw for Joins (Venn-style, very commonly asked):

```
INNER JOIN      → only the overlap between the two circles
LEFT OUTER JOIN → the whole left circle + the overlap
RIGHT OUTER JOIN → the whole right circle + the overlap
FULL OUTER JOIN → both circles entirely (union)
```

 **GFG Reference [Verified]:** [SQL Outer Join \(LEFT / RIGHT / FULL\)](#)  **GFG Reference [GFG Search]:** [SQL Inner Join](#)

DDL in SQL

```
CREATE TABLE Student (
    roll_no INT PRIMARY KEY,
    name     VARCHAR(50) NOT NULL,
    cgpa     DECIMAL(3,2) CHECK (cgpa >= 0 AND cgpa <= 10),
    dept_id  INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

ALTER TABLE Student ADD COLUMN email VARCHAR(100);
ALTER TABLE Student DROP COLUMN email;
DROP TABLE Student;
TRUNCATE TABLE Student; -- removes ALL rows but keeps the table
structure, faster than DELETE (no per-row logging)
```

Explain **DROP** vs **TRUNCATE** vs **DELETE** (a very common comparison question):

DELETE	TRUNCATE	DROP
DML — removes rows one at a time (can use WHERE)	DDL — removes all rows at once, cannot use WHERE	DDL — removes the entire table (structure + data)
Slower (row-by-row logging), can be rolled back	Faster (minimal logging), often cannot be rolled back in some DBMSs	Removes the table definition permanently
Table structure remains	Table structure remains	Table no longer exists at all

 **GFG Reference [GFG Search]:** [DROP vs TRUNCATE vs DELETE in SQL](#)

UNIT-III: Functional Dependencies, Normalization & Database Integrity

3.1 Functional Dependencies (FD) — Basic Definitions

Definition: A functional dependency $X \rightarrow Y$ (read "X functionally determines Y") holds on a relation R if, for any two tuples t_1 and t_2 in R, whenever $t_1.X = t_2.X$, it must also be true that $t_1.Y = t_2.Y$ — in other words, the value of attribute(s) X **uniquely determines** the value of attribute(s) Y.

Example: In `Student(roll_no, name, dept)`, $roll_no \rightarrow name$ holds, because knowing the roll number uniquely determines the student's name.

 **GFG Reference** [[GFG Search](#)]: [Functional Dependency in DBMS](#)

Trivial vs Non-Trivial Dependencies

- **Trivial FD** — $X \rightarrow Y$ where **Y is a subset of X** (e.g., $\{roll_no, name\} \rightarrow roll_no$). This is always true by definition and gives no new information.
- **Non-trivial FD** — $X \rightarrow Y$ where **Y is NOT a subset of X**. This is the kind of dependency that actually matters for database design.

Closure of a Set of Attributes (X^+)

Definition: The closure of an attribute set X, written X^+ , is the set of *all* attributes that are functionally determined by X, given a set of functional dependencies F.

Algorithm to compute X^+ (write these steps in exam):

1. Start with `result = X`.
2. Repeat: for each FD $A \rightarrow B$ in F, if $A \subseteq result$, then add B to `result`.
3. Continue until no more attributes can be added to `result`.
4. The final `result` is X^+ .

Why closure matters: X^+ is used to test whether a given FD $X \rightarrow Y$ is *implied* by F (it's implied if $Y \subseteq X^+$), and to test whether X is a candidate key of the relation (X is a candidate key if X^+ includes **all** attributes of the relation, and no proper subset of X has this property).

 **GFG Reference** [[GFG Search](#)]: [Attribute Closure in DBMS](#)

Closure of a Set of Functional Dependencies (F^+)

Definition: F^+ is the set of **all** functional dependencies that can be logically derived (implied) from a given set F, computed using **Armstrong's Axioms**.

Armstrong's Axioms (the three inference rules — must-memorize for a "prove using Armstrong's axioms" question):

1. **Reflexivity** — if $Y \subseteq X$, then $X \rightarrow Y$ (a trivial dependency always holds).
2. **Augmentation** — if $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any attribute set Z (adding the same attributes to both sides preserves the dependency).
3. **Transitivity** — if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Additional derived rules (useful for solving FD problems faster, worth mentioning):

- **Union** — if $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- **Decomposition** — if $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
- **Pseudo-transitivity** — if $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$.

 **GFG Reference** [[GFG Search](#)]: [Armstrong's Axioms in Functional Dependency](#)

Irreducible (Canonical/Minimal) Set of Dependencies

Definition: A set of functional dependencies F is called **irreducible (or minimal)** if it satisfies all of the following conditions:

1. Every dependency in F has a **single attribute** on its right-hand side.
2. For every dependency $X \rightarrow A$ in F , removing it from F changes the closure F^+ (i.e., no dependency is redundant/removable).
3. For every dependency $X \rightarrow A$ in F , no attribute can be removed from X without changing F^+ (i.e., the left-hand side is minimal — no *extraneous* attributes).

Explain why we need this: Finding the irreducible set (also used to compute **candidate keys** and to test **dependency preservation** during normalization, [Section 3.3](#)) removes redundant dependencies so that database design decisions are based only on the essential, non-redundant relationships between attributes.

 **GFG Reference** [[GFG Search](#)]: [Canonical Cover of Functional Dependencies](#)

[↑ TOC](#)

3.2 Introduction to Normalization

Definition: Normalization is the systematic process of organizing the attributes and tables of a relational database to **minimize data redundancy** and **avoid update, insertion, and deletion anomalies**, by progressively decomposing relations according to their functional dependencies.

 **GFG Reference** [[GFG Search](#)]: [Introduction of Database Normalization](#)

The Three Anomalies (explain all three with an example — extremely common question)

Consider a single unnormalized table `StudentCourse(roll_no, name, course_id, course_name, instructor)`:

1. **Insertion Anomaly** — you cannot insert data about a new course unless at least one student has already enrolled in it (because `course_id/course_name` only exist as part of a student's row).
2. **Deletion Anomaly** — if the *only* student enrolled in a course is deleted, information about that course (its name, instructor) is lost entirely as a side effect.

3. **Update Anomaly** — if a course's instructor changes, you must update the instructor's name in **every row** where that course appears; missing even one row leaves the data inconsistent.

Explain the fix: Normalization removes these anomalies by decomposing this single table into smaller tables — e.g., separate **Student**, **Course**, and **Enrollment** tables — each governed by its own functional dependencies, so that each fact is stored in exactly one place.

 **GFG Reference** [[GFG Search](#)]: [Insertion, Deletion and Update Anomalies in DBMS](#)

Non-Loss (Lossless) Decomposition

Definition: A decomposition of relation R into R1 and R2 is called **lossless (non-loss)** if, when R1 and R2 are naturally joined back together, the result is **exactly** the original relation R — no spurious (extra, incorrect) tuples are introduced and no information is lost.

Condition for a lossless decomposition (memorize this exact rule): A decomposition of R into R1 and R2 is lossless if and only if:

$$(R1 \cap R2) \rightarrow R1 \quad \text{OR} \quad (R1 \cap R2) \rightarrow R2$$

i.e., the common attributes between R1 and R2 must functionally determine *at least one* of the two decomposed relations completely.

 **GFG Reference** [[GFG Search](#)]: [Lossless Join and Dependency Preserving Decomposition](#)

FD Diagram

Explain: A Functional Dependency (FD) diagram is a visual representation where attributes are drawn as nodes and each functional dependency $X \rightarrow Y$ is drawn as a directed arrow from X to Y — used to visually identify partial and transitive dependencies (the basis for 2NF and 3NF checks below).

Example FD diagram for StudentCourse(roll_no, course_id, name, course_name, instructor):

```

roll_no  ────> name
    |
    └───course_id──> course_name, instructor    (course_id alone
determines these)

```

(roll_no, course_id) together → all attributes

[↑ TOC](#)

3.3 Normal Forms

 **GFG Reference** [[Verified](#)]: [Normal Forms in DBMS \(1NF–5NF\)](#)

First Normal Form (1NF)

Definition: A relation is in 1NF if every attribute contains only **atomic (indivisible)** values — no multi-valued or composite attributes, and no repeating groups.

Example fix: A column `phone_numbers` = "9876543210, 9123456789" violates 1NF; the fix is to either split it into separate columns (`phone1`, `phone2` — not ideal, limited) or move it into a separate table (`roll_no`, `phone_number`) with one row per phone number.

Second Normal Form (2NF)

Definition: A relation is in 2NF if it is already in 1NF, **and** every non-prime attribute (an attribute not part of any candidate key) is **fully functionally dependent** on the *entire* primary key — i.e., there is **no partial dependency** (a non-prime attribute depending on only part of a composite primary key).

Explain when this matters: 2NF is only a meaningful concern when the primary key is **composite** (made of more than one attribute) — if the primary key is a single attribute, partial dependency is impossible by definition, so the relation is automatically in 2NF once it's in 1NF.

Example:

```
Enrollment(roll_no, course_id, student_name, marks)
```

```
Primary key: (roll_no, course_id)
```

```
Problem: student_name depends only on roll_no (part of the key), not on the  
whole (roll_no, course_id)
```

```
→ this is a PARTIAL DEPENDENCY → violates 2NF
```

```
Fix — decompose into:
```

```
Student(roll_no, student_name)
```

```
Enrollment(roll_no, course_id, marks)
```

Third Normal Form (3NF)

Definition: A relation is in 3NF if it is already in 2NF, **and** it has **no transitive dependency** of a non-prime attribute on the primary key — i.e., no non-prime attribute depends on another non-prime attribute (rather than depending directly on the key).

Example:

```
Student(roll_no, dept_id, dept_name)
```

```
Primary key: roll_no
```

```
Problem: roll_no → dept_id → dept_name
```

```
So dept_name depends TRANSITIVELY on roll_no (via dept_id) →  
violates 3NF
```

```
Fix — decompose into:
```

```
Student(roll_no, dept_id)
Department(dept_id, dept_name)
```

Formal 3NF definition (a stronger, more precise version worth quoting for full marks): A relation R is in 3NF if, for every non-trivial FD $X \rightarrow A$ in R , at least one of the following holds: (a) X is a **super key** of R , OR (b) A is a **prime attribute** (part of some candidate key) of R . This precise version handles edge cases the simpler "no transitive dependency" explanation misses.

Dependency Preservation

Definition: A decomposition is said to be **dependency preserving** if every functional dependency in the original set F can still be logically enforced/checked using only the FDs of the individual decomposed relations, **without needing to join them back together**.

Why it matters: If a decomposition is not dependency-preserving, then checking whether an update violates a functional dependency would require an expensive join operation across multiple tables every single time data is modified — dependency preservation avoids this cost.

Boyce-Codd Normal Form (BCNF)

Definition: A relation R is in BCNF if, for every non-trivial functional dependency $X \rightarrow A$ in R , **X must be a super key** of R (this is stricter than 3NF, which additionally allowed A to just be a prime attribute).

BCNF vs 3NF — the key exam comparison:

3NF	BCNF
Allows $X \rightarrow A$ if X is a super key OR A is a prime attribute	Requires $X \rightarrow A$ to have X as a super key always , with no exception for prime attributes
Always achievable with dependency preservation	May sometimes require sacrificing dependency preservation to achieve it
A weaker (more permissive) normal form	A stricter normal form — every relation in BCNF is automatically also in 3NF, but not vice versa

Exam tip: A commonly asked question is "give an example of a relation that is in 3NF but not in BCNF" — this typically involves a relation with **overlapping composite candidate keys**, where a non-superkey determines part of another key.

 **GFG Reference** [[GFG Search](#)]: [BCNF vs 3NF](#)

Multivalued Dependencies and Fourth Normal Form (4NF)

Definition: A multivalued dependency $X \twoheadrightarrow Y$ holds on relation R if, for a given value of X , there is a set of values for Y that is **independent** of the values in the remaining attributes of R (i.e., Y and the rest of the attributes vary independently of each other for a fixed X).

Example: Consider `Employee(emp_id, skill, language)` where an employee's skills and known languages are completely independent of each other. Storing both in one table forces you to list every *combination* of skill $\times$ language for that employee, causing redundancy — this is a multivalued dependency

problem ($\text{emp_id} \twoheadrightarrow \text{skill}$ and $\text{emp_id} \twoheadrightarrow \text{language}$), even though there's no ordinary functional dependency issue.

Definition of 4NF: A relation is in 4NF if it is in BCNF, **and** it has no non-trivial multivalued dependency (other than one where the left-hand side is a super key). The fix is to decompose the relation into two separate tables — one for each independent multivalued attribute (e.g., `Employee_Skill(emp_id, skill)` and `Employee_Language(emp_id, language)`).

 **GFG Reference** [[GFG Search](#)]: [Multivalued Dependency and 4NF in DBMS](#)

Join Dependencies and Fifth Normal Form (5NF)

Definition: A join dependency $* (R_1, R_2, \dots, R_n)$ on relation R holds if R can be losslessly reconstructed by joining $R_1, R_2, \dots, R_n$ back together — this is a generalization of the lossless decomposition concept ([Section 3.2](#)) to **more than two** relations.

Definition of 5NF (also called Project-Join Normal Form, PJNF): A relation R is in 5NF if it is in 4NF, and every join dependency in R is **implied by the candidate keys** of R — meaning R cannot be decomposed further into smaller relations without losing information or without the decomposition being logically forced by the keys already.

Exam tip: 5NF questions are usually conceptual/definitional rather than numerical — focus on being able to state the definition and explain that it deals with situations where a relation can only be losslessly decomposed into **three or more** relations (not decomposable into just two without loss), unlike BCNF/4NF which deal with two-way decompositions.

Quick Normal Forms Summary Table (a great one to memorize as a whole):

Normal Form	Removes	Condition
1NF	Repeating groups / non-atomic values	All attribute values are atomic
2NF	Partial dependency	No non-prime attribute depends on part of a composite key
3NF	Transitive dependency	No non-prime attribute depends on another non-prime attribute
BCNF	Anomalies from non-superkey determinants	Every determinant (LHS of a non-trivial FD) is a super key
4NF	Multivalued dependency redundancy	No non-trivial MVD except where LHS is a super key
5NF	Join dependency redundancy	Every join dependency is implied by candidate keys

[↑ TOC](#)

3.4 Database Integrity

Definition: Database integrity refers to the correctness, consistency, and validity of data stored in a database, enforced through a set of rules called **integrity constraints**.

General Idea & Integrity Rules

Two fundamental integrity rules apply to **every** relational database (a must-know pair of definitions):

1. **Entity Integrity Rule** — no attribute that is part of the **primary key** of a relation can be **NULL**. (Every tuple must be uniquely, unambiguously identifiable.)
2. **Referential Integrity Rule** — a **foreign key** value must either match an existing primary key value in the referenced (parent) relation, or be entirely **NULL** — it can never reference a non-existent row.

 **GFG Reference** [[GFG Search](#)]: [Entity Integrity and Referential Integrity Constraints in DBMS](#)

Domain Rules

Definition: Domain constraints restrict the set of legal values that an attribute can take, based on its defined data type and any additional restrictions.

```
cgpa DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10)
```

Attribute Rules (Attribute/Column Constraints)

```
name VARCHAR(50) NOT NULL,  
email VARCHAR(100) UNIQUE,  
age INT CHECK (age >= 18)
```

Explain: These are constraints declared directly on a single column — **NOT NULL** (must have a value), **UNIQUE** (no duplicate values across the column), **CHECK** (a custom boolean condition the value must satisfy), and **DEFAULT** (a value used automatically when none is provided).

Assertions

Definition: An assertion is a general integrity constraint expressed as a predicate that the database must **always** satisfy — unlike a **CHECK** constraint (which applies to a single table/column), an assertion can span **multiple tables**.

```
CREATE ASSERTION total_credit_check  
CHECK (  
    NOT EXISTS (  
        SELECT roll_no FROM Enrollment  
        GROUP BY roll_no  
        HAVING SUM(credits) > 30  
    )  
);
```

Note for exam: Assertions are conceptually important in relational theory but are **not widely supported** by most commercial DBMSs in practice (often implemented instead via triggers) — mentioning this practical limitation shows deeper understanding.

Triggers

Definition: A trigger is a stored procedure that is **automatically executed (fired)** by the DBMS in response to a specified event (**INSERT**, **UPDATE**, or **DELETE**) occurring on a specified table.

```
CREATE TRIGGER before_student_delete
BEFORE DELETE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Audit(roll_no, deleted_on) VALUES (OLD.roll_no,
NOW());
END;
```

Explain the three components of a trigger (a standard definitional question):

1. **Event** — the specific data-modification operation (**INSERT/UPDATE/DELETE**) that causes the trigger to fire.
2. **Condition** — an optional predicate; if present, the trigger's action executes only when this condition is true.
3. **Action** — the sequence of SQL statements that actually execute when the trigger fires (e.g., logging changes, enforcing complex business rules, maintaining derived/redundant data automatically).

Triggers can fire **BEFORE** or **AFTER** the event, and either **FOR EACH ROW** (once per affected row) or **FOR EACH STATEMENT** (once per SQL statement, regardless of how many rows it affects) — this distinction is frequently tested.

 **GFG Reference** [[GFG Search](#)]: [SQL Triggers](#)

Integrity & SQL

```
CREATE TABLE Enrollment (
    roll_no INT,
    course_id INT,
    PRIMARY KEY (roll_no, course_id),
    FOREIGN KEY (roll_no) REFERENCES Student(roll_no)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
);
```

Explain referential integrity actions (ON DELETE/ON UPDATE**) — a common practical question:**

- **CASCADE** — automatically delete/update the matching rows in the child table when the referenced parent row is deleted/updated.

- **SET NULL** — set the foreign key column to **NULL** in the child table instead of deleting/blocking.
- **RESTRICT / NO ACTION** — reject (disallow) the delete/update on the parent if matching child rows still exist.

 **GFG Reference** [**GFG Search**]: [SQL ON DELETE CASCADE / SET NULL](#)

[↑ TOC](#)

UNIT-IV: Transactions, Concurrency & Recovery

4.1 Transactions — Basic Concept & ACID Properties

Definition: A transaction is a sequence of one or more database operations (reads/writes) that is treated as a **single, indivisible logical unit of work** — it must either complete entirely or have no effect at all.

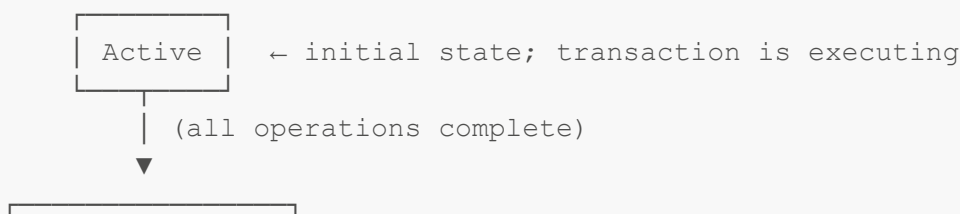
ACID Properties (the single most important definition set in this entire course — must be flawless)

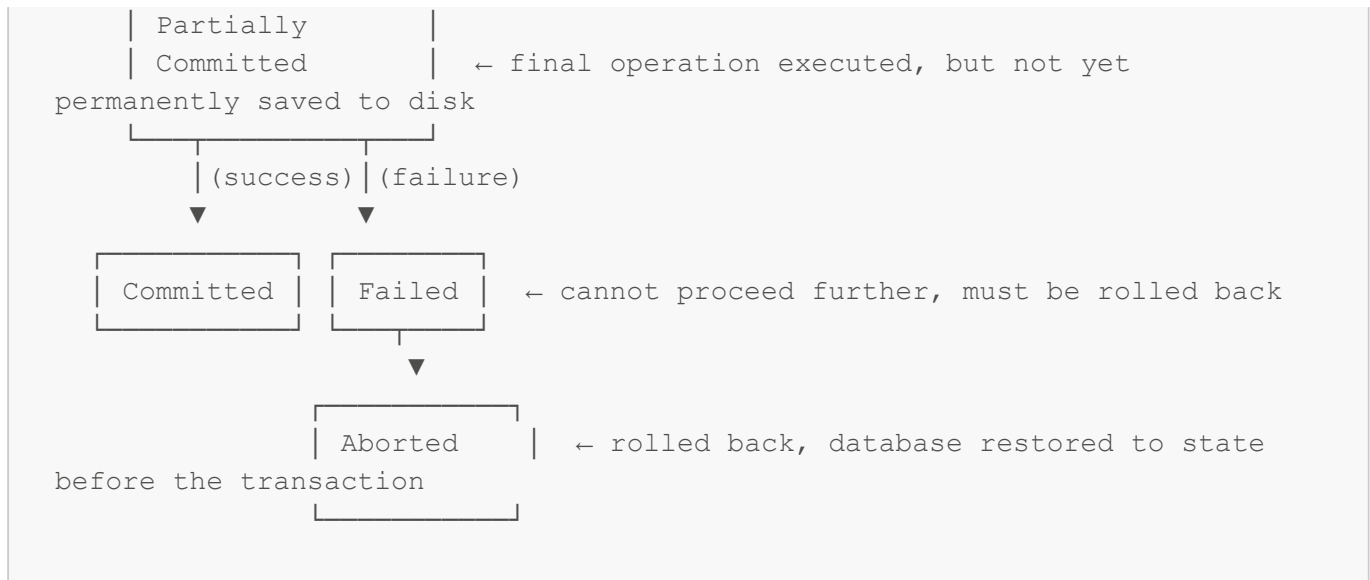
Property	Meaning
Atomicity	A transaction is treated as a single "all or nothing" unit — either all of its operations are reflected in the database, or none are. If a transaction fails partway, all its effects so far must be undone (rolled back).
Consistency	A transaction, when executed alone on a database that starts in a consistent state, must leave the database in a consistent state — i.e., it must not violate any integrity constraints.
Isolation	Even though multiple transactions may execute concurrently , each transaction must appear to execute as if it were the <i>only</i> transaction running — the intermediate (partial) state of one transaction must not be visible to another.
Durability	Once a transaction commits successfully, its changes must persist permanently in the database, even in the event of a subsequent system crash or power failure.

Exam tip: For a 5-mark "explain ACID" question, give the definition of each property PLUS one small example each (e.g., Atomicity → a bank transfer must either debit and credit both accounts, or neither).

 **GFG Reference** [**Verified**]: [ACID Properties in DBMS](#)

Transaction States





Explain the flow in words: A transaction starts in the **Active** state as it executes its operations. Once its final statement executes, it enters **Partially Committed** — the changes are computed but not yet guaranteed to be permanently written to stable storage. If this final write succeeds, it moves to **Committed** (durable, done); if any operation fails at any point, it enters the **Failed** state and must be rolled back to **Aborted**, restoring the database to its state before the transaction began. From Aborted, the transaction can optionally be **restarted** (if the failure was not due to a logical/internal error) or simply killed.

 **GFG Reference** [[GFG Search](#)]: [Transaction States in DBMS](#)

Implementation of Atomicity and Durability

Explain the two dominant approaches (a common "how is atomicity implemented" question):

1. **Shadow-copy / Shadow-paging scheme** — the entire database (or the modified pages) is copied to a new location before any change is made; all updates are made to this shadow copy, and only when the transaction commits does a pointer get atomically switched to make the shadow copy the new current copy. If the transaction fails, the original copy (untouched) is simply used, and the shadow copy is discarded.
2. **Log-based recovery (Write-Ahead Logging, WAL)** — the most widely used approach in practice: before any change is made to the database itself, a record describing that change is first written to a **stable log** on disk. If a crash occurs, the log is used to redo committed changes or undo uncommitted ones (see [Section 4.9](#)).

The Write-Ahead Log (WAL) rule (critical, must be stated exactly): *A log record for a database update must be written to stable storage BEFORE the corresponding database modification itself is written to disk.* This ensures that if a crash occurs right after the data is modified but before the transaction commits, the log still has enough information to undo that partial change.

 **GFG Reference** [[GFG Search](#)]: [Write-Ahead Logging \(WAL\) in DBMS](#)

[↑ TOC](#)

4.2 Concurrent Execution

Definition: Concurrent execution refers to multiple transactions being processed simultaneously (interleaved) by the DBMS, rather than strictly one after another, in order to improve system throughput and resource (CPU/disk) utilization.

Why concurrency is needed (explain briefly): Running transactions one at a time (serially) would badly under-utilize the CPU and disk, since a transaction is often idle (e.g., waiting for disk I/O) — interleaving lets the DBMS work on another transaction during that idle time, improving overall throughput and reducing average waiting time.

Problems Caused by Uncontrolled Concurrency (very frequently asked — list all with examples)

1. **Lost Update Problem** — two transactions read the same data item, and both later write back an updated value; the second write **overwrites** the first, so the first transaction's update is silently lost.
2. **Dirty Read (Temporary Update / Uncommitted Dependency) Problem** — a transaction reads a value written by another transaction that has **not yet committed**; if that other transaction later aborts, the first transaction has read a value that never actually "existed" in the database.
3. **Incorrect Summary (Inconsistent Retrieval) Problem** — one transaction is in the middle of updating several data items (e.g., transferring funds between accounts) while another transaction is simultaneously computing an aggregate (e.g., **SUM**) over those same items, resulting in an incorrect/inconsistent summary value.
4. **Unrepeatable Read Problem** — a transaction reads the same data item twice, but gets a **different** value the second time because another transaction updated and committed a change to that item in between.

 **GFG Reference** [[GFG Search](#)]: [Concurrency Problems in DBMS \(Lost Update, Dirty Read, etc.\)](#)

[↑ TOC](#)

4.3 Serializability

Definition: A concurrent (interleaved) schedule of transactions is called **serializable** if it produces the same final result on the database as *some* serial (one-at-a-time, non-interleaved) execution of those same transactions — serializability is the primary correctness criterion for concurrent schedules.

Types of Serializability:

- **Conflict Serializability** — a schedule is conflict-serializable if it can be transformed into a serial schedule by repeatedly **swapping non-conflicting operations** (two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a **write**). Tested practically using a **precedence graph**: draw an edge $T_i \rightarrow T_j$ if a conflicting operation of T_i occurs before a conflicting operation of T_j on the same data item — the schedule is conflict-serializable **if and only if this precedence graph has no cycle**.
- **View Serializability** — a more general (less restrictive) notion: a schedule is view-serializable if it is "view equivalent" to some serial schedule (same initial reads, same "who writes the final value" for each data item, same read-from relationships). **Every conflict-serializable schedule is also view-serializable, but not vice versa.**

Exam tip: The precedence-graph method for testing conflict serializability, with a step-by-step example schedule, is one of the most commonly asked *numerical* questions in this unit — practice drawing the graph and checking for cycles.

 **GFG Reference** [[GFG Search](#)]: [Conflict Serializability & Precedence Graph in DBMS](#)

[↑ TOC](#)

4.4 Concurrency Control — Basic Idea

Definition: Concurrency control refers to the set of protocols/mechanisms a DBMS uses to ensure that concurrent transaction execution results in a serializable (and thus correct) schedule, while maximizing concurrency.

Main categories of concurrency control protocols (list these, one line each — the detailed protocols are often covered in a follow-up unit but the categories themselves are commonly asked here):

1. **Lock-based protocols** — transactions must acquire locks (shared/exclusive) on data items before reading/writing them, and release them at appropriate points (e.g., **Two-Phase Locking, 2PL**, where a transaction acquires all needed locks in a "growing phase" before releasing any in a "shrinking phase").
2. **Timestamp-based protocols** — each transaction is assigned a unique timestamp at the start; conflicting operations are ordered strictly according to these timestamps, ensuring serializability without needing explicit locks.
3. **Validation (Optimistic) based protocols** — transactions execute freely and are only checked for conflicts at the time of commit ("validation phase"); useful when conflicts are rare, avoiding the overhead of locking for the common case.
4. **Multiversion concurrency control (MVCC)** — the DBMS keeps multiple versions of each data item, so that read operations can be served an appropriate older version without blocking concurrent write operations.

 **GFG Reference** [[GFG Search](#)]: [Concurrency Control in DBMS \(Lock-based, Timestamp, 2PL\)](#)

[↑ TOC](#)

4.5 Deadlock — Basic Idea

Definition: A deadlock is a situation where two or more transactions are each waiting for a lock held by one of the others, forming a **cycle of waiting**, such that none of them can ever proceed.

Example:

T1 holds a lock on A, waiting for a lock on B (held by T2)

T2 holds a lock on B, waiting for a lock on A (held by T1)

→ T1 waits for T2, and T2 waits for T1 → DEADLOCK (circular wait)

Two general approaches to handling deadlock (list both — standard question):

1. **Deadlock Prevention** — design the system so that a deadlock can *never* occur in the first place, e.g., by requiring transactions to acquire all their locks at once, or by using timestamp-ordering schemes (like *Wait-Die* and *Wound-Wait*) to decide whether a transaction should wait or be rolled back.
2. **Deadlock Detection and Recovery** — allow deadlocks to occur, but periodically construct a **wait-for graph** (an edge $T_i \rightarrow T_j$ means T_i is waiting for a lock held by T_j) and check for cycles; if a cycle is found, a deadlock exists, and the system recovers by choosing a "victim" transaction to abort and roll back, releasing its locks for the others.

 **GFG Reference** [[GFG Search](#)]: [Deadlock in DBMS \(Prevention, Detection, Wait-Die/Wound-Wait\)](#)

[↑ TOC](#)

4.6 Failure Classification

Definition: A failure is any event that prevents a transaction from executing correctly, or the entire system from continuing normal operation. Failures are classified by their cause and scope:

1. **Transaction failure** — a single transaction cannot continue its normal execution, due to either:
 - **Logical error** — the transaction can no longer continue due to an internal condition (e.g., bad input, data not found, resource limit exceeded).
 - **System error** — the system enters an undesirable state (e.g., deadlock) and the transaction is aborted by the DBMS to allow the system to recover.
2. **System crash** — a hardware/software fault (e.g., a bug in the DBMS code, a power failure) causes the *entire system* to stop, but the contents of non-volatile storage (disk) are assumed to remain intact and uncorrupted.
3. **Disk failure** — a disk block loses its data, either due to a physical read/write malfunction (a "head crash") or due to a failure during a data-transfer operation.

 **GFG Reference** [[GFG Search](#)]: [Failure Classification in DBMS](#)

[↑ TOC](#)

4.7 Storage Structure Types & Stable Storage Implementation

Storage types classified by volatility:

- **Volatile storage** — does *not* survive a system crash (e.g., main memory/RAM, cache).
- **Nonvolatile storage** — survives a system crash, but may still be lost in a disk failure/disaster (e.g., disk, tape).
- **Stable storage** — an idealized form of storage that is assumed to survive **any** kind of failure, including disk failure — implemented in practice by replicating the same information across several physically separate nonvolatile storage media (e.g., **RAID** — mirroring data across multiple disks), so that the failure of one copy does not lose the data, since the other copies remain intact.

 **GFG Reference** [[GFG Search](#)]: [Storage Types & RAID in DBMS](#)

[↑ TOC](#)

4.8 Data Access (Buffer Management Basics)

Explain briefly: Data resides permanently on disk, organized into fixed-size units called **blocks**. To perform a database operation, a block must first be copied ("input") into a memory buffer; after modification, it is written ("output") back to disk. Each transaction keeps a *local* copy of the data items it works on in memory, and only "output"s the block back to disk at specific points — the precise timing of this disk output relative to the transaction's commit point is exactly what determines how atomicity/durability must be implemented via logging (Section 4.9).

 **GFG Reference** [[GFG Search](#)]: [Buffer Management in DBMS](#)

[↑ TOC](#)

4.9 Recovery & Atomicity: Log-Based Recovery

Definition: Log-based recovery maintains a sequential, append-only **log** on stable storage, containing a record for every update made to the database, which is used to restore the database to a consistent state after a crash.

Typical log record fields (list these — common question): transaction ID, data item name, old value (before the update), new value (after the update) — plus special records for `<Ti start>`, `<Ti commit>`, and `<Ti abort>`.

Recovery procedures using the log (define both — very frequently asked pair):

- **UNDO(Ti)** — restores the value of every data item updated by transaction Ti back to its **old value**, used for transactions that had **not committed** at the time of the crash.
- **REDO(Ti)** — sets the value of every data item updated by Ti to its **new value**, used for transactions that **had committed** before the crash but whose changes might not have been physically written to disk yet.

 **GFG Reference** [[GFG Search](#)]: [Log-Based Recovery in DBMS](#)

Deferred Database Modification

Definition: In this scheme, a transaction's updates are recorded **only in the log** during execution; the actual database itself is modified **only after the transaction commits**, using the log's "new value" entries.

Explain the implication: Since the database is never touched until after commit, **UNDO is never needed** for this scheme — only REDO is required during recovery (for committed transactions whose updates might not have made it to disk yet before the crash), which simplifies the recovery process considerably, at the cost of requiring more memory to hold all updates until commit.

Immediate Database Modification

Definition: In this scheme, updates are applied to the database **immediately** as the transaction executes (before it commits), but the corresponding log record (with both old and new values) must be written to stable storage **before** the actual database modification, following the Write-Ahead Logging rule.

Explain the implication: Because uncommitted changes may already be reflected in the database itself, recovery needs **both UNDO** (to roll back the effects of transactions that had not committed at crash time) **and REDO** (to reapply the effects of transactions that had committed but might not be fully reflected on disk).

Checkpoints (a related and commonly-asked concept): To avoid having to search through the *entire* log all the way back to the very first transaction after a crash (which becomes prohibitively slow over time), the DBMS periodically takes a **checkpoint** — writing all currently buffered log records and modified data to stable storage, then writing a `<checkpoint>` record to the log. During recovery, the system only needs to consider transactions that were active at or after the most recent checkpoint, since everything before it is guaranteed to be safely on disk.

 **GFG Reference** [[GFG Search](#)]: [Checkpoints in DBMS](#)

[↑ TOC](#)

4.10 Query Processing and Optimization

Definition: Query processing is the sequence of steps a DBMS takes to translate a high-level query (SQL) into an efficient, low-level sequence of operations that can be executed on the physical database to produce the result.

 **GFG Reference** [[GFG Search](#)]: [Query Processing in DBMS](#)

Steps in Query Processing (a standard diagram/list question)

1. Parsing & Translation
 - SQL query is parsed for syntax, then translated into an internal representation,
typically an expression tree in relational algebra
2. Optimization
 - The optimizer generates multiple equivalent execution plans (since relational algebra expressions can be rewritten in many equivalent forms) and estimates the COST of each
(based on statistics like table size, available indexes, selectivity of conditions),
then picks the plan with the LOWEST estimated cost
3. Evaluation
 - The chosen "query execution plan" is actually executed by the query-evaluation engine,
which invokes low-level operators (e.g., specific join algorithms, specific access methods like index scan vs full table scan) to produce the final result

Explain Query Optimization further (frequently the focus of the exam question): Because a single SQL query can typically be expressed as many *logically equivalent* relational algebra expressions (e.g., pushing a σ (selection) down before a join versus after it), and because different low-level algorithms exist for the same operation (e.g., nested-loop join vs. hash join vs. sort-merge join), the query optimizer's job is to search this large space of equivalent plans and choose the one with the lowest estimated execution cost — typically dominated by estimated **disk I/O cost**.

Key optimization heuristics (mention these for extra marks):

- **Perform selection (σ) and projection (π) operations as early as possible** — reduces the size of intermediate relations before more expensive operations (like joins) are performed.
- **Combine Cartesian products with a subsequent selection into a join** — a join is generally implemented far more efficiently than computing a full Cartesian product and then filtering it.
- **Choose the most selective conditions first**, and use available indexes wherever possible, to minimize the number of tuples processed at each stage.

 **GFG Reference** [[GFG Search](#)]: [Query Optimization in DBMS](#)

[↑ TOC](#)

UNIT-V: Graph, Spatial, Temporal Databases & NoSQL

5.1 Graph Databases

Definition: A graph database is a type of NoSQL database that stores data as a **graph structure** — consisting of **nodes** (representing entities), **edges** (representing relationships between entities), and **properties** (key-value attributes attached to nodes and/or edges) — designed specifically to make traversing relationships fast and natural.

Why use a graph database (explain the motivation, common question): In a relational database, representing and querying highly interconnected data (e.g., "find all friends-of-friends up to 3 levels deep" in a social network) requires multiple expensive **JOIN** operations, and the number of joins grows with the depth of the relationship being queried. Graph databases store relationships as **first-class citizens** (as actual edges, physically linking nodes), so traversing a relationship is a fast, direct pointer-following operation, regardless of how deep the traversal goes — this is often called **"index-free adjacency."**

Key concepts:

- **Node** — represents an entity (e.g., a **Person**, a **Product**), can have labels (types) and properties.
- **Edge (Relationship)** — a directed, named connection between two nodes (e.g., **(Person) -[:FRIENDS_WITH]->(Person)**), can also carry its own properties (e.g., **since: 2020**).
- **Property Graph Model** — the most common graph data model, where both nodes and edges can hold arbitrary key-value properties.

Example query language mention: Popular graph databases include **Neo4j** (using the query language **Cypher**), and standard query languages like **Gremlin** and **SPARQL** (for RDF graphs) are also widely used.

```
Example Cypher query (Neo4j):  
MATCH (p:Person)-[:FRIENDS_WITH]->(friend)  
WHERE p.name = 'Rohit'  
RETURN friend.name;
```

Use cases (list a few for a "applications of graph databases" question): Social networks (friend recommendations), fraud detection (finding unusual transaction patterns/rings), recommendation engines, network/IT infrastructure management, and route/navigation optimization.

 **GFG Reference** [[GFG Search](#)]: [Graph Databases \(NoSQL\)](#)

[↑ TOC](#)

5.2 Spatial Databases

Definition: A spatial database is a database optimized to store, query, and manipulate **spatial (geometric/geographic) data** — such as points, lines, and polygons representing real-world locations and shapes — along with specialized indexing and query operations tailored to spatial relationships.

Key data types (typically supported via an extension like PostGIS on PostgreSQL, or built into some DBMSs):

- **Point** — a single location (e.g., a GPS coordinate: latitude/longitude).
- **Line (LineString)** — a sequence of connected points (e.g., a road, a route).
- **Polygon** — a closed shape representing an area (e.g., a country's boundary, a building footprint).

Spatial operations/queries (list these — common exam question):

- **Distance queries** — find the distance between two spatial objects (e.g., "how far is this restaurant from my current location?").
- **Range queries** — find all objects within a given area (e.g., "find all hospitals within 5 km").
- **Nearest-neighbor queries** — find the k closest objects to a given point (e.g., "find the 3 nearest petrol pumps").
- **Topological queries** — test spatial relationships like **contains**, **intersects**, **overlaps**, **touches** (e.g., "which district contains this point?").

Spatial indexing (explain why ordinary B-tree indexes don't work well here — an important conceptual point): A standard B-tree index works well for one-dimensional, ordered data (like numbers or strings), but spatial data is inherently **multi-dimensional**, so specialized index structures are used instead:

- **R-tree** — groups nearby spatial objects using bounding rectangles, arranged hierarchically, so a search can quickly discard entire regions that clearly don't overlap the query area.
- **Quad-tree** — recursively divides 2D space into four quadrants, used for indexing point/region data.

Use cases: GIS (Geographic Information Systems), navigation/mapping apps (Google Maps), urban planning, environmental monitoring, ride-hailing/logistics applications.

 **GFG Reference** [[GFG Search](#)]: [Spatial Databases \(R-tree, Quad-tree Indexing\)](#)

[↑ TOC](#)

5.3 Temporal Databases

Definition: A temporal database is a database that manages **time-varying data** — it keeps track not just of the current state of data, but also of when facts were true, allowing queries about the past (and sometimes even planned future) states of the data.

Two important types of time to distinguish (a classic, must-know comparison for this topic):

- **Valid time** — the time period during which a fact is true **in the real world** (e.g., the period an employee actually held a particular job title), independent of when it was recorded in the database.
- **Transaction time** — the time period during which a fact was stored **in the database** as the current, "officially known" value (i.e., when it was actually recorded/committed, and when — if ever — it was superseded/corrected).

Explain the distinction with an example (a great way to answer a definition question): Suppose an employee's salary increase, effective from January 1st, is only entered into the database on January 15th. The **valid time** for the new salary begins January 1st (when it became true in reality), but the **transaction time** begins January 15th (when the database actually recorded it) — a temporal database can maintain and query both timelines independently.

Classification of database types based on which time dimension they track:

- **Historical database** — supports only *valid time*.
- **Rollback database** — supports only *transaction time* (lets you see the database exactly as it was recorded at any past point).
- **Bitemporal database** — supports **both** valid time and transaction time simultaneously — the most complete (and most complex) form.

Use cases: Financial auditing (regulatory requirement to know exactly what data looked like at a past point), insurance/legal record-keeping, medical history tracking, and version-controlled data systems.

 **GFG Reference** [[GFG Search](#)]: [Temporal Database — Valid Time vs Transaction Time](#)

[↑ TOC](#)

5.4 NoSQL Databases

Definition: NoSQL ("Not Only SQL") refers to a broad class of database systems that depart from the traditional fixed-schema, table-based relational model, designed instead for **flexible/dynamic schemas**, **horizontal scalability** (adding more machines rather than a bigger single machine), and handling large volumes of unstructured or semi-structured data.

 **GFG Reference** [[GFG Search](#)]: [NoSQL Databases — Types and Examples](#)

Why NoSQL — Motivation (CAP Theorem)

Definition (CAP Theorem): In a distributed data system, it is impossible to simultaneously guarantee all three of the following; at most **two out of three** can be fully achieved at any given time:

- **Consistency (C)** — every read receives the most recent write (or an error).
- **Availability (A)** — every request receives a (non-error) response, even if it isn't the most recent data.
- **Partition Tolerance (P)** — the system continues to operate even if network communication is lost between nodes (a network partition).

Explain the practical implication (a commonly asked follow-up): Since network partitions are unavoidable in any real distributed system, a distributed database must effectively choose between prioritizing **Consistency** (a "CP" system — may reject requests during a partition to avoid returning stale data) or **Availability** (an "AP" system — always responds, but might occasionally return slightly stale/inconsistent data). Most NoSQL databases favor Availability and Partition tolerance (**AP**), trading strict consistency for a weaker guarantee called **eventual consistency** (all replicas will converge to the same value *eventually*, once updates finish propagating), whereas traditional relational databases traditionally favor strict Consistency.

 **GFG Reference** [[GFG Search](#)]: [CAP Theorem in DBMS](#)

Types of NoSQL Databases (the core classification — list all 4 with an example of each, extremely likely exam question)

Type	Data Model	Example Databases	Best Suited For
Key-Value Store	Simple key → value pairs, value is opaque to the database	Redis, DynamoDB, Riak	Caching, session storage, simple lookups requiring extreme speed
Document Store	Stores semi-structured "documents" (typically JSON/BSON), each document can have a different structure	MongoDB, CouchDB	Content management, catalogs, applications with evolving/nested data
Column-Family (Wide-Column) Store	Data organized into column families rather than fixed rows — rows can have varying columns; optimized for very wide, sparse tables	Apache Cassandra, HBase	Large-scale analytics, time-series/logging data, write-heavy workloads
Graph Database	Nodes and edges (as covered in Section 5.1)	Neo4j, ArangoDB	Highly interconnected data — social networks, recommendations

SQL vs NoSQL Comparison (a favourite direct comparison question — memorize this table)

SQL (Relational)	NoSQL
Fixed, predefined schema	Dynamic/flexible schema — different records can have different fields
Data organized as tables with rows/columns	Data organized as key-value pairs, documents, wide columns, or graphs, depending on type

SQL (Relational)	NoSQL
Scales vertically (adding more power — CPU/RAM — to a single server)	Scales horizontally (adding more servers/nodes to a cluster)
Strong (ACID) consistency, favors correctness	Often favors eventual consistency (BASE model) and availability
Best for structured data with complex relationships and transactions	Best for large volumes of unstructured/semi-structured data, high-velocity writes, and horizontal scaling
Query language: SQL (standardized)	No single standard query language — varies by product (e.g., MongoDB Query Language, Cassandra's CQL)

BASE properties (explain as the NoSQL counterpart to ACID — a strong pairing to bring up in the same answer):

- **Basically Available** — the system guarantees availability of data, prioritizing responsiveness over strict correctness.
- **Soft state** — the state of the system may change over time, even without new input, as data propagates/converges across replicas.
- **Eventual consistency** — the system will *eventually* become consistent once all updates have propagated, given enough time without new writes.

 **GFG Reference** [[GFG Search](#)]: [SQL vs NoSQL & ACID vs BASE](#)

Example: Document Store (MongoDB-style)

```
// A single "document" in a MongoDB-style collection — note the flexible,
nested structure
{
  "_id": "101",
  "name": "Rohit",
  "dept": "CSE",
  "skills": ["Flutter", "Node.js", "PostgreSQL"],
  "address": {
    "city": "Nagpur",
    "state": "Maharashtra"
  }
}
```

Explain why this is powerful (contrast with relational, common follow-up): In a relational model, **skills** (multi-valued) would require a separate junction table (as covered in [Section 3.3's 1NF discussion](#)), and **address** (composite) would either be flattened into columns or split into another table. A document database allows nesting arrays and sub-objects *directly inside a single record*, matching how application code (e.g., JSON objects) naturally represents this data — avoiding joins for data that is always accessed together.

[↑ TOC](#)

Quick Exam-Day Revision Checklist

Before the exam, make sure you can write these from memory (they cover the highest-weightage recurring questions):

- ☐ [Advantages of DBMS over file-processing](#) (all 7 points)
- ☐ [Three-schema architecture](#) diagram + physical vs logical data independence
- ☐ [Types of keys](#): super, candidate, primary, alternate, foreign (with definitions)
- ☐ [Weak vs Strong entity sets](#) + example
- ☐ [Specialization vs Generalization vs Aggregation vs Inheritance](#)
- ☐ [Rules for reducing ER schema to tables](#) (all 6 cases: strong entity, weak entity, 1:1, 1:M, M:N, specialization)
- ☐ [Relational Algebra operators](#) (σ , π , $\cup$, $-$, $\times$, $\bowtie$) with symbols and examples
- ☐ [Relational Algebra vs Relational Calculus](#) comparison table
- ☐ [WHERE vs HAVING](#); [correlated vs non-correlated subquery](#)
- ☐ [Types of SQL JOINS](#) with Venn diagrams
- ☐ [DELETE vs TRUNCATE vs DROP](#)
- ☐ [Armstrong's Axioms](#) (Reflexivity, Augmentation, Transitivity)
- ☐ [Insertion/Deletion/Update anomalies](#) with example
- ☐ [1NF \$\rightarrow\$ 2NF \$\rightarrow\$ 3NF \$\rightarrow\$ BCNF \$\rightarrow\$ 4NF \$\rightarrow\$ 5NF](#) (condition for each, in order)
- ☐ [3NF vs BCNF comparison](#)
- ☐ [Entity Integrity Rule vs Referential Integrity Rule](#)
- ☐ [Triggers](#): event-condition-action, BEFORE/AFTER, ROW/STATEMENT
- ☐ [ACID properties](#) (all four, with one example each)
- ☐ [Transaction state diagram](#) (Active $\rightarrow$ Partially Committed $\rightarrow$ Committed/Failed $\rightarrow$ Aborted)
- ☐ [Lost Update, Dirty Read, Incorrect Summary, Unrepeatable Read problems](#)
- ☐ [Conflict serializability via precedence graph](#) (practice a numerical example)
- ☐ [Deadlock: prevention vs detection-and-recovery](#)
- ☐ [Deferred vs Immediate database modification](#) (UNDO/REDO requirement)
- ☐ [Checkpoints](#) — why they're needed
- ☐ [Steps in query processing](#): Parsing $\rightarrow$ Optimization $\rightarrow$ Evaluation
- ☐ [CAP theorem](#) (Consistency, Availability, Partition tolerance) + why P is non-negotiable
- ☐ [4 types of NoSQL databases](#) with one example DB each
- ☐ [SQL vs NoSQL comparison table](#)
- ☐ [ACID vs BASE](#)
- ☐ [Valid time vs Transaction time](#) (temporal databases)
- ☐ [Graph database: nodes, edges, index-free adjacency](#)
- ☐ [Spatial database: R-tree/Quad-tree indexing, spatial query types](#)

[↑ TOC](#)

CS-6418: Advanced Database Management System / M.Tech Exam-Ready Notes / Units I–V / GFG links added for quick reference